Why the CFU1 driver doesn’t use Windows 98’s own USB stack

cfu1-win9x project note - RATOC REX-CFU1 / SL811HS - July 2026

The question: Windows 98 ships a full USB stack (USBD.SYS, USBHUB.SYS, HID and storage class drivers). If our card could present itself as a host controller at the bottom of that stack, hubs, keyboards, mice and mass storage would all work "for free". Why does this project implement its own parallel USB stack instead?

What "being a real host controller" means on Win98

```
USBSTOR / HID class / vendor class drivers
        USBHUB.SYS  (hub + device enumeration)
        USBD.SYS    (the USB bus driver)
        UHCD.SYS or OPENHCI.SYS  (host controller drivers)
        [PCI UHCI/OHCI silicon]
```

Everything above USBD is documented WDM. If we could plug in at the bottom - as a host controller driver (HCD) for the SL811 - the entire class-driver ecosystem would come for free.

1. The bottom edge is undocumented

USBD and the HCDs talk over a private interface ("HCDI"). It is not in the DDK, has no headers, no samples, and changed between releases. USBD and UHCD were effectively co-developed as one unit that happens to be two files. Writing an SL811 HCD means reverse-engineering that interface from disassembly of USBD.SYS - a bigger RE project than this entire driver so far, against a moving target across 98, 98SE and ME.

2. The hardware model doesn’t fit

UHCI and OHCI are schedule-based DMA controllers: the OS builds frame lists and transfer descriptors in memory and the silicon walks them autonomously. The SL811HS is a one-transaction-at-a-time PIO chip with a 240-byte buffer. Even with HCDI fully decoded, an SL811 HCD would be emulating a DMA schedule engine in software, transaction by transaction, under an interface that assumes the hardware does the pacing. Linux pulled this off (sl811-hcd) because Linux’s HCD interface was designed to admit oddball controllers; Win9x’s wasn’t designed to admit anyone.

3. History agrees

No third party ever shipped a 9x HCD. Companies that needed USB on unsupported hardware (OrangeWare and others) shipped complete parallel stacks - their own bus driver, hub logic and class drivers - which is precisely the architecture this project ended up with. That is not coincidence; it is everyone hitting the same wall.

4. This card is doubly cursed

Before USB even starts, the REX-CFU1’s CIS lies about its configuration register (declared at attribute 0xF8, really at 0xFC), so Windows’ own PCMCIA layer cannot configure the card at all. Any solution begins with this driver’s COR-at-0xFC hack -

Microsoft’s stack would never even power the card up.

How did supported controllers get drivers? They didn’t

Nobody outside Microsoft ever wrote a Win9x host controller driver - not even the silicon vendors. Microsoft inverted the problem: instead of documenting the software interface (HCDI) so vendors could write drivers, they standardised the hardware interface so vendors didn’t have to. There were exactly two blessed register-level specs: UHCI (Intel’s, implemented by Intel and VIA silicon, driven by Microsoft’s own UHCD.SYS) and OHCI (the open Compaq/Microsoft/National spec implemented by everyone else, driven by Microsoft’s own OPENHCI.SYS). A controller implementing one of those register layouts bit-for-bit was run by the in-box driver; the vendor never touched the stack - "driver development" was a hardware compliance exercise. Anything else simply did not exist on Windows. There was no privileged program and no NDA’d HCDI kit, because nobody was ever supposed to need one. The same model continues today (EHCI for USB 2.0, xHCI for USB 3: one spec, one Microsoft driver, zero vendor HCDs).

That is exactly how the SL811 fell through the crack: it is not a UHCI or OHCI implementation but an embedded host controller - a PIO register-file chip designed for microcontrollers and PDAs, where the OS vendor hand-writes a custom HCD per platform. That is why RATOC could ship a Windows CE driver (CE’s USB stack had a real, documented HCD porting layer - odd silicon was expected) but never a Windows 98 one: on the PC, the front door only opened for two shapes of key, and this card is the wrong shape. The REX-CFU1 was never "missing a driver" - it was architecturally locked out of PC Windows from the day it was designed. The parallel stack is not a workaround for a missing driver; it is the only citizenship this card could ever hold.

The honest ledger

Is the undocumented-IOS swamp (drive letters) actually cheaper than the undocumented-HCDI swamp? A fair challenge. The score so far: the IOS route has already produced a mounting, reading, writing drive letter, and is down to one known translation bug plus stabilisation. The HCDI route starts at zero, requires disassembling Microsoft binaries rather than probing live structures (a much slower feedback loop), and still ends at the hardware-model mismatch even if the RE succeeds - while reusing almost none of what has been built.

Where the instinct is right: every layer climbed (VOLTRACK semantics, VRPs, criteria translation) is a re-implementation of things Microsoft’s stack does for free. On Windows 2000, with its documented miniport model, the HCD route would win outright. On 9x, the parallel stack remains the pragmatic path - and it is genuinely near its summit. The HCDI idea stays on the roadmap as a research item: the right dream, the wrong swamp for a weekend.